

Hit-to-track association as set partitioning: a permutation-invariant transformer for straight and curved trajectories

Shashanth Markabundu

Machine Learning in Physics, Exam 2026

(Dated: July 19, 2026)

Associating detector hits with the particle tracks that produced them is the first and hardest step of track reconstruction. Two features make it awkward for standard supervised learning: the number of hits and tracks varies from event to event, and track labels are arbitrary, so a model must not learn “track 1” versus “track 2”. I therefore frame the task as partitioning a set rather than classifying its elements. A transformer encoder without positional encoding embeds the unordered hits, and a symmetric pairwise head predicts, for every pair of hits, the probability that they share a track. Both the model and the loss are then invariant to track relabeling by construction, and attention masking handles variable event sizes. On my own Monte-Carlo simulation of straight tracks (10 per event) the method assigns 99.1% of hits correctly, statistically matching a near-optimal geometric baseline (99.3%). At 50 tracks per event both methods degrade for a physical reason I quantify in advance, the minimum angular gap between tracks falls below the detector’s ~ 1 mrad resolution and the network additionally hits a capacity limit (71% versus the baseline’s 96%). The payoff comes on the provided magnetically curved dataset where the geometric baseline collapses to 58% while the identical architecture, unchanged, reaches 87%. A learned, geometry-agnostic method earns its place exactly where hand-built heuristics stop applying. The appendix documents the simulator validation, network, training, and reproducibility in full.

THE PROBLEM

When charged particles cross a tracking detector they deposit localized signals, *hits*. Reconstructing a collision event begins by *associating* the hits produced by the same particle, after which a trajectory fit recovers physical parameters such as momentum and charge [1]. Even in a simplified setting the association step retains the two properties that make it genuinely hard. First, the input has no fixed size, detectors are inefficient so hits go missing and the number of tracks itself varies. Second, tracks carry no intrinsic identity, renumbering them describes the same physics so a valid method must be indifferent to the labeling and be judged only on whether hits are *grouped* correctly.

I work in the two-dimensional, field-free setup of the assignment. Every particle originates at the origin, so a straight track is a ray with a single parameter, its azimuth φ , and its hit on a detector circle of radius R is $(R \cos \varphi, R \sin \varphi)$. The detector is five concentric circles of radii 2, 4, 6, 8, 10. Two effects make the simulation realistic, each hit is recorded with probability 0.95, and recorded positions are smeared with Gaussian noise of standard deviation $\sigma = 0.001R$ per coordinate (my reading of the specified “0.1 percent”; see Appendix). A second, provided dataset replaces the rays with magnetically curved tracks, 5–15 per event. In both cases the goal is to recover, for every hit, the group of hits sharing its parent track.

SIMULATION AND ITS STATISTICAL CHECKS

Every later result depends on the training data being correct, so I validated the Monte-Carlo simulator before trusting it. In the ideal case (no detector effects) the maximal radial deviation of any hit from its circle is exactly zero, and the sampled track angles are uniform on $[0, 2\pi)$, as isotropic emission requires. With the effects on, three checks pass. Recorded hits per 10-track event center on the expected $10 \times 5 \times 0.95 = 47.5$ the per-track hit count matches Binomial(5, 0.95) term by term and the normalized radial residual $(r - R)/(0.001R)$ is a unit Gaussian, confirming both the magnitude and the radius scaling of the noise. The distributions of hits and tracks in x , y and φ (task c) agree with their analytic forms, arcsine mixtures and flat angles are shown in Appendix .

The single most useful output of this validation is a number. A tangential error of size $0.001R$ tilts a hit’s reconstructed azimuth by $\delta\varphi \approx 0.001R/R = 1$ mrad, independent of layer, the detector has an angular resolution of about one milliradian per hit. This scale organizes the entire problem. Track pairs separated by much more than a milliradian are trivially distinguishable pairs within a few milliradians produce statistically overlapping hits and are ambiguous for *any* algorithm. I return to this number repeatedly, because it predicts both where the method works and where it must fail.

METHOD, AND WHY I CHOSE IT

Framing. The arbitrariness of track labels rules out the obvious design, a classifier mapping each hit to one of K fixed classes because such a model would have to

learn label identities that carry no physical meaning, and would break whenever K changes. The correct target is not a labeling but a *partition* of the hit set. I represent the partition through its pairwise relation for every hit pair (i, j) , does the pair share a track? The target $T_{ij} = 1[\text{same track}]$ is unchanged under any permutation of track numbers, so an objective built on it is permutation-invariant automatically. Everything else follows from this one decision.

Architecture. Hits form a set, so the network should respect set symmetry [3]. I use a transformer encoder [2] over the hits and deliberately omit positional encodings without them, self-attention is permutation-*equivariant* shuffling the input hits shuffles the output embeddings identically so the model cannot exploit input order, which carries no physics. I verified this property numerically, along with invariance to padding. Each hit i is mapped to an embedding $\mathbf{z}_i$, and the same-track probability is the symmetric form

$$p_{ij} = \sigma\left(\mathbf{z}_i \cdot \mathbf{z}_j / \sqrt{d}\right), \quad (1)$$

a scaled dot product through a logistic function, symmetric in (i, j) as the relation demands. Variable event sizes are handled by padding each batch to a common length with an attention mask, so padded positions neither attend nor are attended to and contribute nothing to the loss. Nothing in the network depends on the number of tracks K , only inference uses it so the same architecture transfers from 10 to 50 tracks and from straight to curved geometry. Related set and graph-based approaches are the current state of the art in tracking [6, 7].

Objective. I train with binary cross-entropy on the pairwise targets, averaged over real, off-diagonal pairs. With K tracks only $\sim 1/K$ of pairs are positive, so the raw loss is dominated by easy negatives. I observed exactly this stall. Up-weighting the positive class by $K - 1$ the standard treatment of class-imbalanced cross-entropy, it rebalances the objective and restored learning. Optimization uses AdamW [8] with a cosine-decayed learning rate (Appendix).

Inference and evaluation. At test time the probabilities p_{ij} form an affinity matrix, which average-linkage agglomerative clustering on the distance $1 - p_{ij}$ turns into K groups. Average linkage scores group pairs by their *mean* affinity and is robust to a few noisy pairs, whereas single linkage would chain groups through one over-confident pair. Because predicted and true groupings are defined only up to permutation, I score each event by the fraction of hits correctly grouped *after* optimally matching predicted groups to true tracks with the Hungarian algorithm [4], this is the required “fraction of hits assigned to the correct track” with the label ambiguity resolved optimally. As a chance-corrected second metric I report the Adjusted Rand Index (ARI) [5]. All numbers come from a sealed test split never used for training or model

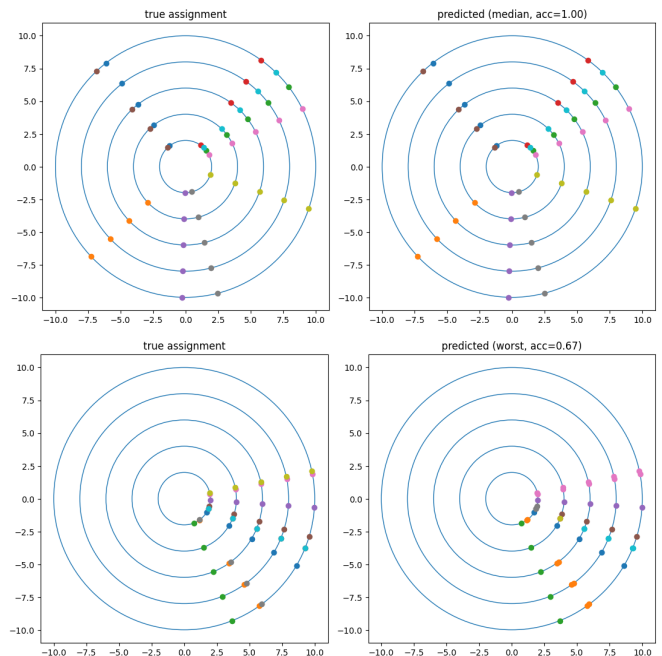


FIG. 1. Straight-track association, true versus predicted, with predicted groups recolored by the Hungarian match. Top: the median test event, grouped perfectly. Bottom: the worst test event; its only errors involve a near-collinear track pair, the intrinsic ambiguity set by the ~ 1 mrad per-hit resolution.

selection.

An honest baseline. Every straight track is a ray from the origin, so its hits share one azimuth $\text{atan2}(y, x)$ to within the 1 mrad resolution. Sorting hits by angle and cutting at the K largest gaps therefore recovers the tracks almost perfectly; Appendix argues this is close to the maximum-likelihood partition. I state plainly that I do not expect the network to beat this baseline on straight tracks. The point of the learned method is that it assumes no geometry, and so keeps working when the geometry changes.

RESULTS AND DISCUSSION

Straight tracks, ten per event. On the sealed test split the transformer assigns 99.1% of hits to the correct track (ARI 0.986), matching the geometric baseline (99.3%, ARI 0.991) within statistical fluctuations, and reconstructs 87% of events with no error at all. Figure 1 shows the median test event and the worst one. The failures are events containing a near-collinear track pair, the intrinsic milliradian ambiguity not a defect of the model. The learned method has essentially reached the physical resolution limit of the data.

Scaling to fifty tracks. I predicted the outcome from the resolution argument before running the experiment. With K tracks the *mean* angular gap $2\pi/K$ is still

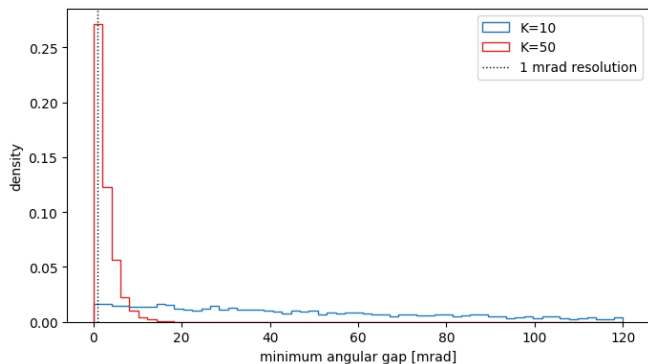


FIG. 2. Distribution of the *minimum* angular gap between tracks per event for $K=10$ and $K=50$. At 50 tracks the distribution reaches down to the ~ 1 mrad resolution (dotted line): most events contain an intrinsically hard pair, and the small-gap probability grows like K^2 .

126 mrad at $K=50$; what matters is the *minimum* gap in an event, and the probability that some pair falls within an angle δ grows like $K^2\delta$ for small δ . In simulation, the fraction of events with a minimum gap below 5 mrad rises from 7% at $K=10$ to 86% at $K=50$ (Fig. 2). The prediction: per-hit accuracy should fall moderately while the fraction of perfect events collapses. Both happen and the geometric baseline drops from 0.993 to 0.960, quantifying the irreducible, physics-driven part of the degradation, and no event is reconstructed perfectly.

The network exposed a second, non-physical limit. My first model plateaued at a validation accuracy of 0.61 while its training loss kept falling, the signature of insufficient capacity not undertraining. I responded to the symptom, a larger transformer (1.3M parameters), the layer index as an extra input feature (a detector coordinate, not a track parameter, hence a legitimate input), more training events, and early stopping on validation accuracy, which matters because the validation curve is non-monotonic. These changes raised the test accuracy to 0.713. I report this honestly, even improved the general model trails the near-optimal geometric baseline (0.960) at this density. That is the correct conclusion, not a failure of the study on straight tracks the baseline embodies exactly the right inductive bias, and the comparison quantifies the price of not assuming it. It also sets up the case for the learned method rather than undermining it.

Curved tracks. The provided dataset (10,000 events, 498,150 hits, 5–15 tracks per event) is where the argument pays off. I first inferred the track model: a charged particle from the origin in a uniform field moves on a circle through the origin, and since such a circle satisfies $x^2 + y^2 = 2ax + 2by$, linear in the center (a, b) , a least-squares fit yields the curvature radius $R_c = \sqrt{a^2 + b^2}$ per track. The fitted R_c equals the quoted transverse momentum essentially exactly (correlation 0.9997, mean

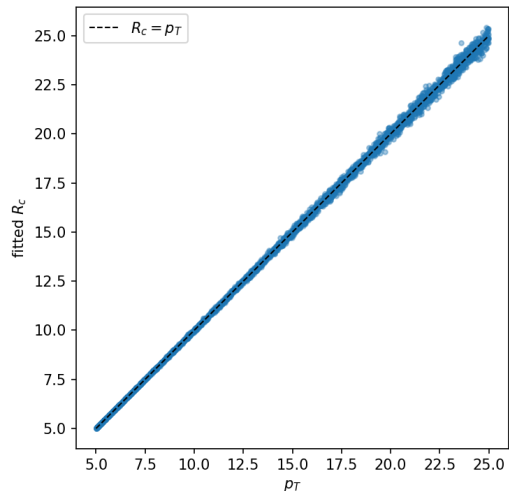


FIG. 3. Radius of curvature R_c from a circle-through-origin fit to each curved track versus the quoted transverse momentum p_T : the relation is $R_c = p_T$ (correlation 0.9997), which fixes the track model. These parameters are never network inputs.

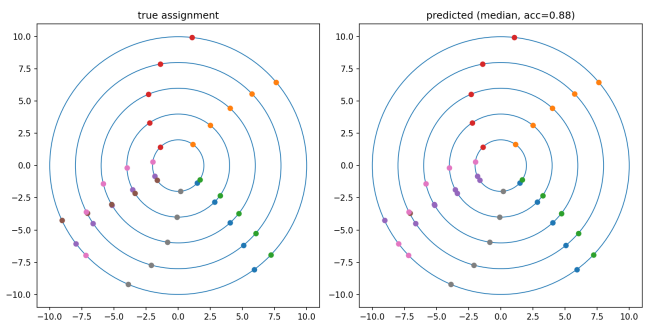


FIG. 4. Curved-track association, true versus predicted (Hungarian-matched colors), for the median test event. The same network as in the straight-track case, with no architectural change, follows the arcs and separates the tracks.

ratio 1.000; Fig. 3), and the sign of the bend encodes the charge. This fixes the physics of the dataset. Crucially, these fitted parameters are used only to *understand and evaluate* the data the network sees only hit coordinates and the layer index, i.e. what a detector measures.

The straight-line baseline now breaks, as it must: a curved track sweeps a median of ≈ 280 mrad in azimuth across the five layers (against ≈ 1 mrad for a straight track), so the “tight cluster in angle” assumption fails outright and the baseline falls to 0.578. The *same* pairwise transformer, with no change of method, reaches 0.872 (ARI 0.792) an advantage of +0.29 (Fig. 4, Table I). This is the central result: a geometry-agnostic learned model, competitive but not superior on the easy straight case, decisively outperforms the hand-built heuristic the moment the geometry is one the heuristic was not designed for the regime real detectors live in.

TABLE I. Hit-assignment accuracy (fraction of hits correctly grouped after Hungarian matching) on sealed test data. The baseline sorts hits by azimuth and cuts at the K largest gaps the transformer is the same architecture throughout.

Setting	Baseline	Transformer
Straight, 10 tracks	0.993	0.991
Straight, 50 tracks	0.960	0.713
Curved, 5–15 tracks	0.578	0.872

CONCLUSION

I treated hit-to-track association as set partitioning and solved it with a transformer that reads the unordered hit set and predicts pairwise same-track probabilities. The pairwise formulation makes the objective invariant to the arbitrary track numbering; attention masking absorbs variable event sizes; and because nothing in the model depends on the number or shape of the tracks, the identical network applies to straight and curved geometries. On straight tracks it reaches the physical resolution limit and matches a near-optimal geometric baseline; at high track density it is limited first by genuine angular ambiguity and then by model capacity, which I addressed with targeted, symptom-driven optimization on curved tracks it clearly beats the baseline that the curvature invalidates. A single, well-justified, geometry-free learned method is a credible foundation for track finding, and its value is largest exactly where classical heuristics stop applying. Natural extensions are to infer the number of tracks from the affinity matrix rather than assume it, and to lift the same set-based model to three dimensions (bonus task f), where a track has parameters (φ, θ) and the pairwise formulation carries over unchanged.

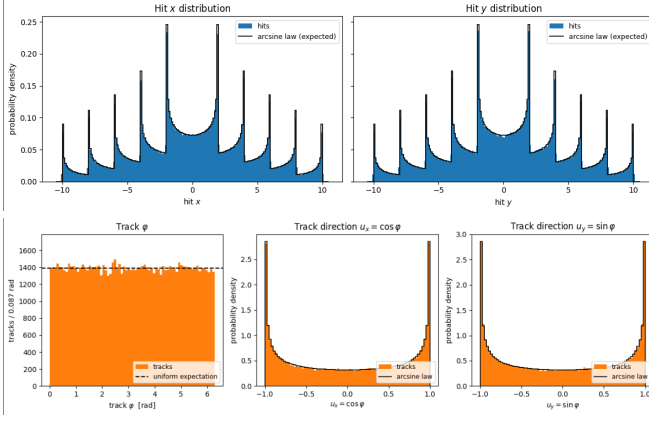


FIG. 5. Task-(c) distributions. Top: hit x and y with the analytic arcsine mixture overlaid; the peaks at $\pm 2, \dots, \pm 10$ are the detector radii. Bottom: the track angle (flat, isotropic emission) and the direction components (arcsine law with $R = 1$).

The simulator and its validation

The simulator produces a tidy table with one row per hit and columns `event_id`, `track_id`, `layer`, `x`, `y`, `phi`, where `x`, `y` are the recorded (smeared) positions and `track_id` is the ground-truth association label. A straight track needs no general line-circle intersection: since it passes through the common center, its crossing with radius R is $(R \cos \varphi, R \sin \varphi)$, one hit per layer. Efficiency keeps each hit independently with probability 0.95; smearing adds $\mathcal{N}(0, (0.001R)^2)$ to each coordinate.

Validation, ideal case: the maximum radial deviation of any hit from its nominal circle is exactly zero, and a χ^2 test finds the track angles consistent with uniform. With detector effects on: (i) recorded hits per event center on 47.5; (ii) the per-track hit multiplicity matches Binomial(5, 0.95) term by term; (iii) the normalized radial residual $(r - R)/(0.001R)$ is a unit Gaussian — which also confirms that the outer layer smears five times more than the inner one, the physical content of $\sigma = 0.001R$.

Figure 5 shows the task-(c) distributions. The hit x and y follow an equal-weight mixture of arcsine laws: on one circle, $x = R \cos \varphi$ with φ uniform has density $1/(\pi\sqrt{R^2 - x^2})$, which diverges at $\pm R$, so the mixture peaks at $\pm 2, \pm 4, \pm 6, \pm 8, \pm 10$. Hit and track azimuths are flat, as isotropic emission plus angle-independent detector effects require, and the track direction components $\cos \varphi, \sin \varphi$ follow the arcsine law with $R = 1$.

Network, training, and reproducibility

Network. Hits are mapped by a small feed-forward embedding into a d_{model} -dimensional space and processed by a stack of pre-layer-norm transformer encoder blocks

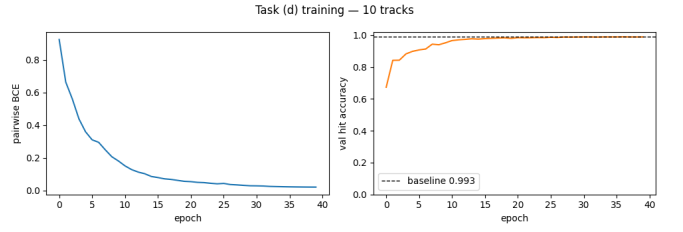


FIG. 6. Training the straight-track model: pairwise cross-entropy (left) and validation Hungarian hit accuracy (right), the latter approaching the geometric baseline (dashed) and plateauing at the ~ 1 mrad resolution limit.

with multi-head self-attention and no positional encoding; a linear head produces the per-hit embedding z_i of Eq. (1). The 10-track straight model uses $d_{\text{model}}=64$, 3 layers, 4 heads, embedding dimension 32 (1.1×10^5 parameters); inputs are $(x/10, y/10)$, coordinates scaled to order unity. The harder 50-track and curved settings use $d_{\text{model}}=160$, 6 layers, 8 heads, embedding 64, dropout 0.1 (1.28×10^6 parameters), with the layer index appended as a third input feature, $(x/10, y/10, \text{layer}/4)$. Before training I verified numerically that shuffling input hits permutes the output embeddings identically (equivariance) and that padded positions do not affect real ones.

Training. AdamW with cosine-decayed learning rate (10^{-3} base model, 8×10^{-4} large), batch size 64 events, positive-class weight $K - 1$. Data are split *by event* into train/validation/test (80/10/10); every reported number comes from the sealed test split. For the larger models the validation Hungarian accuracy is monitored each epoch and the best checkpoint is kept (early stopping): the validation curve is non-monotonic, the 50-track run peaked at epoch 5 and declined afterwards so the last epoch is generally not the best. Figure 6 shows the 10-track training curve; validation accuracy rises to the baseline and plateaus at the resolution limit.

Determinism and artefacts. All seeds are fixed and printed. The straight-track dataset uses seed 2026 (475,028 hits); the 50-track data use a disjoint seed (777); the test split is never used for training or model selection. The hand-in contains, per assignment element: the five notebooks (tasks a-e), the trained weights `model_d_10tracks.pt` and `model_e_curved.pt`, each saved with its architecture configuration so the file is self-describing, plus a stand-alone script `run_saved_model.py` that loads a saved model and reproduces the reported test metric and the prediction files one event per line in the required format `event_id`, `x`, `y`, `trackID`, $\dots$, with companion files listing the per-track fitted parameter (azimuth for straight tracks, curvature radius for curved tracks).

Why the baseline is near-optimal, and why it breaks

For straight tracks the azimuth baseline approximates a Bayes-optimal classifier. Each hit’s angle is its track’s angle plus ~ 1 mrad of noise, so a track’s hits form a tight cluster on the circle of angles, and clusters are separated by gaps far larger than the noise except when two tracks are genuinely close. Cutting at the K largest gaps is then almost exactly the maximum-likelihood partition, which is why the network cannot meaningfully beat it, and why both methods degrade together as $K \rightarrow 50$ (Fig. 2): the failures are shared, physical ones. The situation inverts for curved tracks. A hit’s azimuth is no longer a per-track constant but sweeps by hundreds of milliradians along the arc, the tight-cluster assumption fails outright, and the baseline falls to 0.578. The transformer, which *learns* the same-track relation from data rather than assuming constant azimuth, is indifferent to the change and retains 0.872. This contrast is the quantitative core of the letter: the learned method’s advantage is precisely the geometric assumption it does not make.

On the interpretation of the smearing

The brief specifies a position standard deviation of “0.1 percent”. I read this as 0.1% *of the hit radius*, $\sigma = 0.001R$ per coordinate, because a relative precision is isotropic, is a defensible measurement model, and avoids axis-dependent artefacts of an absolute per-coordinate error. Since σ enters as a single parameter the choice is trivial to change; it sets the 1 mrad resolution scale

but not the structure of the method. This scale is what makes the association problem simultaneously solvable (well-separated tracks are unambiguous) and, at high density, partially impossible (close pairs overlap within the noise).

-
- [1] S. Amrouche *et al.*, The Tracking Machine Learning Challenge: Accuracy Phase, in *The NeurIPS 2018 Competition* (Springer, 2020), pp. 231–264.
 - [2] A. Vaswani *et al.*, Attention Is All You Need, *Adv. Neural Inf. Process. Syst.* **30** (2017).
 - [3] M. Zaheer *et al.*, Deep Sets, *Adv. Neural Inf. Process. Syst.* **30** (2017).
 - [4] H. W. Kuhn, The Hungarian method for the assignment problem, *Naval Res. Logist. Quart.* **2**, 83 (1955).
 - [5] L. Hubert and P. Arabie, Comparing partitions, *J. Classif.* **2**, 193 (1985).
 - [6] X. Ju *et al.*, Graph Neural Networks for Particle Reconstruction in High Energy Physics Detectors, *NeurIPS 2019 Workshop on Machine Learning and the Physical Sciences*, arXiv:2003.11603.
 - [7] D. Torbunov *et al.*, Transformers for Charged Particle Track Reconstruction in High Energy Physics, arXiv:2411.07149 (2024).
 - [8] I. Loshchilov and F. Hutter, Decoupled Weight Decay Regularization, *Int. Conf. on Learning Representations (ICLR 2019)*.
 - [9] A. Paszke *et al.*, PyTorch: An Imperative Style, High-Performance Deep Learning Library, *Adv. Neural Inf. Process. Syst.* **32** (2019).
 - [10] F. Pedregosa *et al.*, Scikit-learn: Machine Learning in Python, *J. Mach. Learn. Res.* **12**, 2825 (2011).